

CMPS 2200 Assignment 2

In this assignment we'll work on applying the methods we've learned to analyze recurrences, and also see their behavior in practice.

As with previous assignments, some of your answers will go in `main.py`. Please add your written answers to `answers.md` which you can convert to a PDF using `convert.sh`. Alternatively, you may scan and upload written answers to a file names `answers.pdf`.

1. Derive asymptotic upper bounds for each recurrence below. (2 pts ea.)

a) $T(n) = 2T(n/3) + 1$

Enter answers in answers.md

.
.
.

b) $T(n) = 5T(n/4) + n$

Enter answers in answers.md

.
.
.

c) $T(n) = 7T(n/7) + n$

Enter answers in answers.md

.
.
.

d) $T(n) = 9T(n/3) + n^2$

Enter answers in answers.md

.
.
.

e) $T(n) = 8T(n/2) + n^3$

Enter answers in answers.md

.
.
.

f) $T(n) = 49T(n/25) + n^{3/2} \log n$

Enter answers in answers.md

.
.
.

g) $T(n) = T(n-1) + 2$

Enter answers in answers.md

.
.
.

h) $T(n) = T(n - 1) + n^c$, with $c \geq 1$

Enter answers in answers.md

·
·
·

i) $T(n) = T(\sqrt{n}) + 1$

For this recurrence, you may assume that n is a power of 2 and that the recurrence will end when $n = 2$.

Enter answers in answers.md

·
·
·

2. Suppose that for a given task you are choosing between the following three algorithms:

- Algorithm $\mathcal{A}$ solves problems by dividing them into five subproblems of half the size, recursively solving each subproblem, and then combining the solutions in linear time.
- Algorithm $\mathcal{B}$ solves problems of size n by recursively solving two subproblems of size $n - 1$ and then combining the solutions in constant time.
- Algorithm $\mathcal{C}$ solves problems of size n by dividing them into nine subproblems of size $n/3$, recursively solving each subproblem, and then combining the solutions in $O(n^2)$ time.

What are the asymptotic running times of each of these algorithms? (3 pts ea.) Which algorithm would you choose? (3 pts)

Enter answers in answers.md

3. Now that you have some practice solving recurrences, let's work on implementing some algorithms. In lecture we discussed a divide and conquer algorithm for integer multiplication. This algorithm takes as input two n -bit strings $x = \langle x_L, x_R \rangle$ and $y = \langle y_L, y_R \rangle$ and computes the product xy by using the fact that $xy = 2^n x_L y_L + 2^{n/2}(x_L y_R + x_R y_L) + x_R y_R$.

Use the stub functions in `main.py` to implement two algorithms for integer multiplication: a divide and conquer algorithm that runs in quadratic time, and the Karatsuba-Ofman algorithm running in subquadratic time. Then test the empirical running times across a variety of inputs to test whether your code scales in the manner described by the asymptotic runtime. (20 pts)